Checkpoint #3 Plan – Library App (Group 6)

Oct 8, 2026 · @Josh bossio

Overview

Checkpoint #3 is due **Tuesday, October 27, 2026**: a hosted web app with login, data entry pages and queries/reports, plus at least 2 database triggers, emailed as a link with access details to the TAs and uramamur@bcm.edu. The goal is to send it **Monday, October 26**, so the 27th is only a safety day.

Area	Owner (proposed)	What it covers
A. Accounts & Access	Juan Manuel	Login, logout, roles (member vs. staff), account pages
B. Catalog Admin	Adan	Add, edit and withdraw catalog items, copies, authors, publishers
C. Search & Holds	Farwa	Search the catalog, place and cancel holds, "my holds"
D. Circulation	Austin	Checkout, return, renew, short vs. standard loans
E. Fines, Notifications & Reports	Josh	Fines, notifications, the 3 reports, plus Azure hosting and the skeleton

Four checkpoints keep everyone aligned:

- **Wed Oct 14:** the skeleton is live on Azure and login works.
- **Wed Oct 21:** every area's pages are merged and deployed.
- **Sat Oct 24:** code freeze. Only bug fixes after this.
- **Mon Oct 26:** final test as each role, then send the email.

What each person builds

Each role owns three things for its area: the SQL, the backend routes (the server code behind each page), and the pages people see. Everyone works with Node.js, Express, the mysql2 driver and plain HTML/CSS/JavaScript. The details below say what to build, which tables you touch, and what "done" looks like.

A. Accounts & Access (Juan)

You decide who is allowed in and what they can see. Everyone else's pages depend on your login, so the first part must be finished in Week 1.

Tables: Member, Staff, MemberType.

Build, in this order:

1. **Login and logout.** The user types a username or email and a password. The server finds the row in Member or Staff and compares the password to `Password_hash` using a hashing library such as bcrypt. Never store or compare a plain password. A successful login starts a session.
2. **Two small checks the others reuse.** One says "you must be logged in", the other says "you must be staff". Teammates attach them to their pages, so publish them early in the repo.
3. **Staff forms.** Register a new member, edit a member's record, and deactivate an account by setting `Is_active` to false. Never delete a member.
4. **Member "my account" page.** Shows the member's own details and member type.

SQL you write: find a user by username; insert a member; update `Is_active`; list members with their type name (a JOIN to MemberType). **Query:** members by type.

Watch out for: deactivated accounts must not log in; an employee who also borrows has both a Staff row and a Member row linked by `Staff_id`; the sample data needs hashed passwords, not typed ones.

Done when: a member and a staff test account both log in and see different menus, and a member who types a staff-only address is turned away.

B. Catalog Admin (Adan)

You build the staff-only forms that keep the library's catalog up to date: items, their copies, publishers and authors.

Tables: CatalogItem, the category tables (Book, Videogame, BoardGame, DVD, Music, Newspaper, ResearchPaper), BookAuthor, PaperAuthor, Publisher, Author, Copy.

Build, in this order:

1. **Item list page.** Shows each item with its category, publisher and number of copies (a JOIN to Publisher, and a count of Copy rows).
2. **Add-item form.** The user picks a category, and the form shows that category's extra fields (an ISBN for a book, a platform for a videogame). Saving is two inserts that must succeed together: the CatalogItem row first, then the category row, with `Item_type` written explicitly in both. Use a transaction so one never saves without the other.
3. **Edit item.** Same fields, updating both rows.
4. **Copies.** Add a copy (unique barcode, condition, shelf location). "Withdraw" a copy by setting its `Status` to withdrawn, not by deleting it.
5. **Publishers and authors forms,** and a way to link authors to a book or research paper.

Query: items and their copy counts.

Watch out for: the database refuses a category row whose `Item_type` doesn't match the item, so show a friendly message instead of a raw error; the app must require a replacement cost, with historical newspapers between $150 and $425 and research papers between $174 and $425; a duplicate barcode or ISBN should say so plainly.

Done when: a staff user can add a book with two authors and two copies, edit it, and withdraw one copy, and the list page shows the right counts.

C. Search & Holds (Farwa)

You build how members find items and put their name in line for ones that are out.

Tables: CatalogItem and the category tables (reading), Copy, Hold, Member.

Build, in this order:

1. **Search page.** A box for a title or author, plus a filter for category. Results show each item with how many copies are available.
2. **Availability.** An item's available count is the copies whose `Status` is available, minus any copy reserved for a Ready hold. This number also decides short vs. standard loans, so Austin will reuse your query.
3. **Place a hold.** A button on the item page that adds a Hold row for the logged-in member. The database allows only one open hold (Active or Ready) per member and item, so show a clear message if they already have one.

4. **"My holds" page.** Lists the member's holds with their status, and a cancel button that sets the status to Cancelled.
5. **Staff hold list.** Shows the waiting holds for an item, oldest first by `Hold_date`, so staff know who is next.

SQL you write: a search with LIKE and JOINs; the availability count; insert a hold; update a hold's status. **Query:** availability.

Trigger, with Josh: when a copy is returned and someone is waiting, their hold becomes Ready, the returned copy is recorded in `Reserved_copy_id`, and a Notification is created. A Ready hold must name its copy, or the database refuses it.

Watch out for: a hold is placed on the item, not on one copy; a Ready hold expires after 2 days.

Done when: a member searches, sees correct availability, places a hold, sees it in "my holds", and cancels it.

D. Circulation (Austin)

You build the heart of the library: lending an item, taking it back, and extending a loan. It is the busiest area, so the work starts Oct 15 and Josh pairs with you on the first flow.

Tables: Loan, Copy, Member, MemberType, Hold, and Fine (to check unpaid fines).

Build, in this order:

1. **Checkout form (staff only).** Staff enter a copy's barcode and a member. The server checks the rules before saving: the member is active, is under the total and media loan limits for their member type, has no unpaid fines above $2.50, and the copy is available and not reserved for someone else's Ready hold. If the borrower is a Staff employee, a different staff member must process it.
2. **Pick the loan type.** If only 1 copy is available, it is a short loan (3 days, no renewal). With 2 or more, it is a standard loan (14 days). Save the Loan row with the staff member's id.
3. **Return form.** Record `Return_date` and `Return_staff_id` together. The database requires both or neither.
4. **Renew form.** Standard loans only, within the member type's renewal limit, and not if another member has an Active or Ready hold on the item. A renewal adds 1 to `Renewal_count` and moves the due date out 14 days.
5. **Open loans page.** Lists what a member has out and when it is due.

Trigger: when a loan is opened, the copy's `Status` becomes on loan. When it is returned, it goes back to available. Josh pairs with you on this.

Query: open loans for a member.

Watch out for: the database allows only one open loan per copy and checks the due-date windows; never write to `Open_copy_id`, because MySQL fills it in; media means videogames, DVDs, music and board games, and books, newspapers and papers are not media.

Done when: a full cycle of checkout, renew and return works on the sample data, and each rule above is refused with a clear message when it is broken.

E. Fines, Notifications & Reports (Josh)

You build the money and messages side of the library and the 3 reports, and you also own hosting, the skeleton and the final email.

Tables: Fine, Notification, Loan, Member, plus reads across the whole database for reports.

Build, in this order:

1. **Fine assessment.** When a late return is recorded, add a fine: days late times $0.35, capped at $10 per item. A damaged or lost copy is fined its replacement cost. A loan gets at most one fine of each type.
2. **Fines pages.** A member sees their fines and can pay one (`Is_paid` true with a `Date_paid`). Staff can waive or adjust an amount.
3. **Notifications page.** Lists a member's messages, with a way to mark one as read.
4. **Three reports,** each a formatted, staff-only page built from queries: overdue loans, fines owed per member, and most borrowed items.

Triggers: an overdue fine is created when a late return is recorded, and you pair with Farwa on the hold-ready notification. At least 2 triggers are required across the group.

Also yours: the Azure subscription, MySQL server and web app; the GitHub repo and skeleton; `sample_data.sql` with a test login for each role; and the final email to the TAs.

Watch out for: a fine's paid flag and paid date must agree, and the paid date cannot be before the assessed date; reports need enough sample data to look real.

Done when: returning an overdue item creates the right fine, a member can pay it, a notification appears, and all 3 reports show real rows on the hosted site.

Week 1 (Oct 8 to Oct 14): skeleton and login

By Wednesday, October 14, a bare Node.js app is running on Azure, connected to the hosted MySQL database, with login working. Everyone else builds on that skeleton, so it comes first.

Person	Task this week	Done when
Josh	Create the Azure resource group and MySQL server, load the dump, create the web app, set up the GitHub repo and shared settings. Write `sample_data.sql` for all 20 tables, with test accounts for each role.	The hello-world page is live and reads a row from the hosted database
Farwa	Help build the skeleton: Express layout, base page, navigation. Then start the catalog search page (C).	Search page lists catalog items from sample data
Adan	Help build the skeleton. Then start the catalog list page (B).	Catalog list page shows items with their type and copies
Juan	Account system (A): password hashing, login, logout, member vs. staff roles.	A member and a staff test account can log in and see different menus
Austin	Midterms week, so no coding is expected. Read the loan rules (short vs. standard, renewals, one open loan per copy) and write the checkout steps in plain English.	A one-page list of checkout, return and renew steps that Josh can review

Everyone, this week:

- Install Node.js and GitHub Desktop, clone the repo, and run the app on your own computer.
- Load the project dump into a local MySQL copy, so nobody tests on the shared hosted database.
- Keep database passwords out of GitHub. They go in a local `.env` file that is never committed.

Week 2 (Oct 15 to Oct 21): build every area

By Wednesday, October 21, each person has their pages working, merged into the main branch and deployed. Each area needs data entry forms (add, modify, "delete"), and the group needs 3 queries, 3 reports (formatted documents built from query results) and at least 2 triggers in total.

Area	Pages and forms	Query	Trigger
A. Accounts (Juan)	Add, edit and deactivate members and staff; staff-only pages blocked for members; "my account" page	Members by type	None needed
B. Catalog (Adan)	Add and edit an item with its subtype row (Item_type written explicitly); add copies; withdraw a copy instead of deleting it; publishers and authors forms	Items and their copy counts	None needed
C. Search & Holds (Farwa)	Search by title, author or type with availability; place and cancel a hold; "my holds"	Availability: copies marked available minus copies reserved for a Ready hold	Hold becomes Ready, a Notification is created (with Josh)
D. Circulation (Austin)	Checkout, return and renew forms; short vs. standard loan chosen from available copies	Open loans for a member	Loan opened or returned, Copy.Status follows (Josh pairs)
E. Fines & Reports (Josh)	Assess and pay a fine; notifications list	3 reports, each a formatted page built from queries: overdue loans; fines owed per member; most borrowed items	Overdue fine created when a late return is recorded

Rules for the week:

- **Triggers live in the database.** Write each as its own `.sql` file in the repo, so it can be re-run on Azure.
- **Small merges, often.** Merge into main at least every two days. One person owns each file, so GitHub Desktop merges stay simple.
- **Austin starts coding Thursday, Oct 15.** Circulation is the busiest area, so Josh pairs with him on the first checkout and return flow.
- **Deactivate, never delete,** members, copies and loans. History tables (Loan, Fine, Hold, Notification) keep their rows.

Week 3 (Oct 22 to Oct 26): integrate, freeze, test

No new features after Saturday, October 24. The last days are for testing every page as every role and fixing what breaks.

Date	What happens	Who
Thu Oct 22	Merge everything into main. Resolve conflicts. Deploy and click through each area once.	Everyone
Fri Oct 23	Test every page as a member and as a staff user. Write bugs in a shared list.	Everyone, each testing another person's area
Sat Oct 24	Code freeze. Fix bugs only.	Owners fix their own bugs
Sun Oct 25	Switch the web app to the Basic plan so it stays on. Reset the hosted database to clean sample data. Confirm the test logins.	Josh
Mon Oct 26	Final test from a fresh browser. Send the email to the TAs and uramamur@bcm.edu.	Josh sends, everyone checks the link
Tue Oct 27	Deadline. Safety day only.	

Deployment steps on Azure

One person (Josh) owns the Azure subscription. The steps below take the project from an empty subscription to a public link. Portal menu names can differ slightly, so check each screen as you go.

1. **Resource group.** Create `library-group6-rg` in one region near Houston. Use the same region for everything below.
2. **MySQL server.** Create an Azure Database for MySQL flexible server: MySQL 8.0, Development workload, Burstable B1ms. The admin name cannot be `root` or `admin`. Choose Public access, add your own IP address to the firewall, and allow Azure services to connect.
3. **Load the database.** Connect from Workbench using the server's address and the DigiCert root certificate (Azure requires an encrypted connection). Run the project dump, then `sample_data.sql`, then the trigger files. Check that `SHOW TABLES;` lists 20 tables.
4. **Prepare the Node.js app.** The repo needs a `package.json` (Express, the mysql2 driver), an entry file, and the certificate file. Database host, user, password and name are read from environment variables, never typed into the code.
5. **Create the web app.** App Service, Linux, Node.js runtime, same region. Use the free plan while developing; switch to the Basic plan by October 25 so it stays on.
6. **Set the app's settings.** In the web app's environment variables add `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` and `SECRET_KEY`, and set the startup command to start the app with npm start.
7. **Deploy from GitHub.** In the web app's Deployment Center connect the project repo and the main branch. Each merge to main then redeploys automatically.
8. **Check it.** Open the web app's link, use the log stream if a page fails, and confirm a page reads from the hosted database.
9. **Give teammates access.** In the subscription's Access control, add each teammate's UH email so they can see logs and settings.
10. **Protect the credit.** Set a budget alert on the subscription, and delete the resource group after grading.

The stack in the project notes includes Docker. A Dockerfile can help everyone run the same setup locally, but it is not needed to deploy this way. Ask the professor whether the hosted app must run as a container.

Submission checklist and risks

The email goes out Monday, October 26 with the link, a test login for each role, and the course, section and team number in the subject line, for example `COSC 3380 – Section [__] – Team 6 – Checkpoint #3`.

- [ ] The link opens from a fresh browser with no Azure sign-in
- [ ] Login works for a member test account and a staff test account
- [ ] Every area has working add, modify and "delete" forms
- [ ] At least 3 queries and 3 reports (formatted pages built from query results) work on pages
- [ ] At least 2 triggers exist in the hosted database, and each has been fired once
- [ ] Sample data is in all tables
- [ ] The web app is on the Basic plan and stays running
- [ ] The email lists the link, the test logins and the team members
- [ ] Sent to the TAs and uramamur@bcm.edu

Open questions to answer this week: the course section number, the TAs' email addresses, and whether Docker is required for the hosted app.

Risk	Fallback
Austin has no time before Oct 15	Josh builds the first checkout and return flow and hands it over
Free web plan runs out its daily time	Move to the Basic plan early and set a budget alert
A student region or service is blocked in Azure	Try another region, and keep the database and web app together
A trigger has a bug on the hosted database	Keep every trigger in its own `.sql` file and test it locally first
Merge conflicts in GitHub	One owner per file, small merges, and Josh resolves conflicts on Oct 22